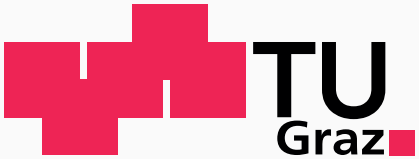


Oblivious Transfer, Extended

Oblivious Evaluations for Privacy

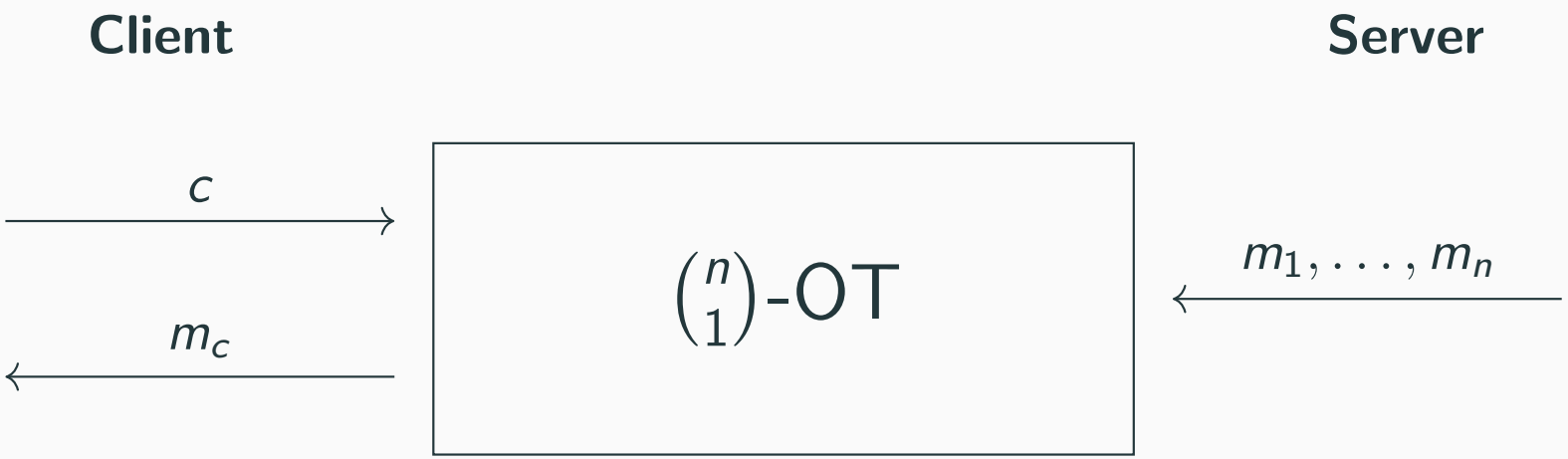
Lena Heimberger

QSCAN 2026, Vienna

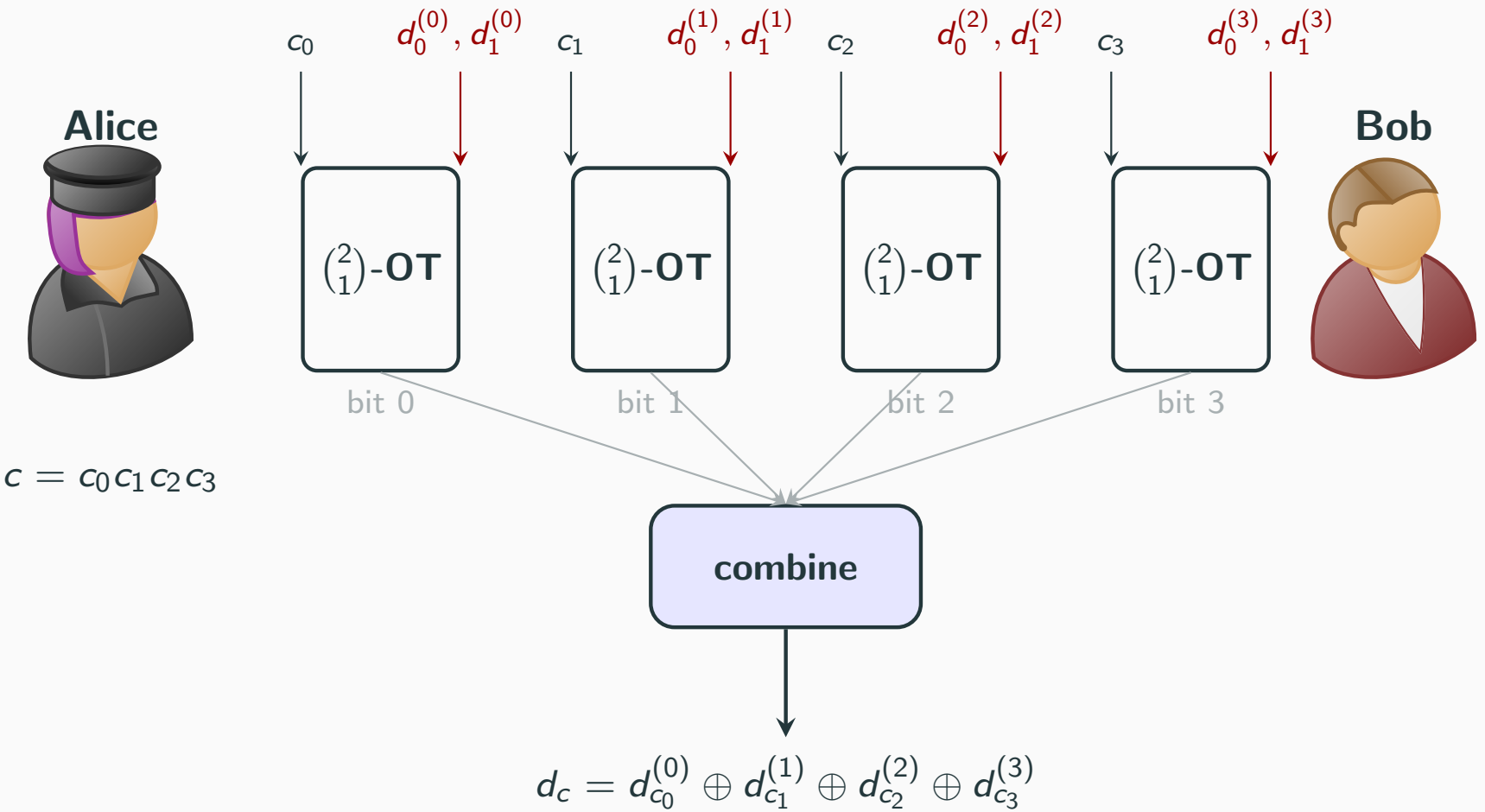
9^{th} of June, 2026

2 Protocols to start the 2-party computation

$\binom{n}{1}$-OT



$\binom{n}{1}$-OT from $\log n$ OTs



Agreeing on a flat

Motivating example

Alice and Bob are looking for a new flat. Depending on the location, they only want to pay a certain amount of rent per room, but neither want to put down concrete numbers.

Agreeing on a flat

Motivating example

Alice and Bob are looking for a new flat. Depending on the location, they only want to pay a certain amount of rent per room, but neither want to put down concrete numbers.

Figure 1:
https://www.aquarell.me/bilder/oesterreich/wien

Agreeing on a flat

Motivating example

Alice and Bob are looking for a new flat. Depending on the location, they only want to pay a certain amount of rent per room, but neither want to put down concrete numbers.

Figure 1:
https://www.aquarell.me/bilder/oesterreich/wien

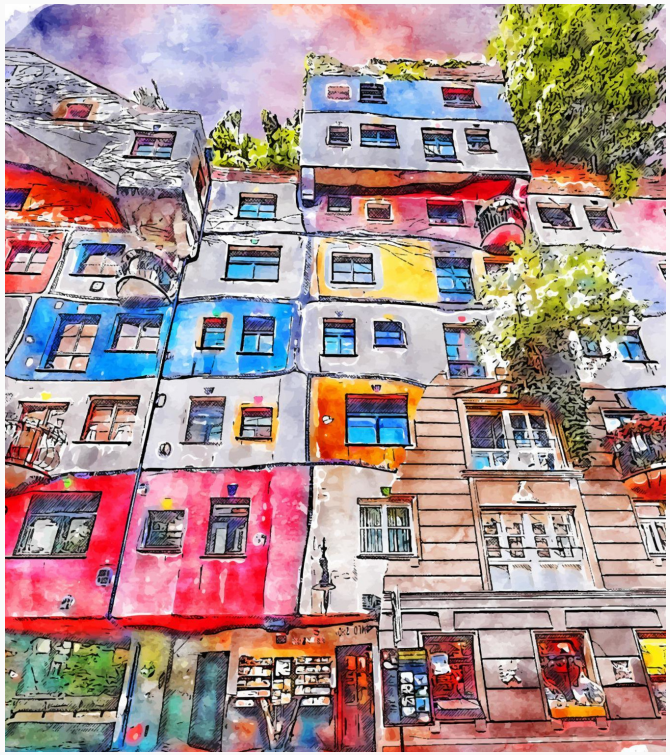


Figure 2:
https://de.vecteezy.com/vektorkunst/12911197-wien-osterreich-aquarell-skizze-handgezeichnete-illustration

Agreeing on a flat

Motivating example

Alice and Bob are looking for a new flat. Depending on the location, they only want to pay a certain amount of rent per room, but neither want to put down concrete numbers.

Figure 1: https://www.aquarell.me/bilder/oesterreich/wien

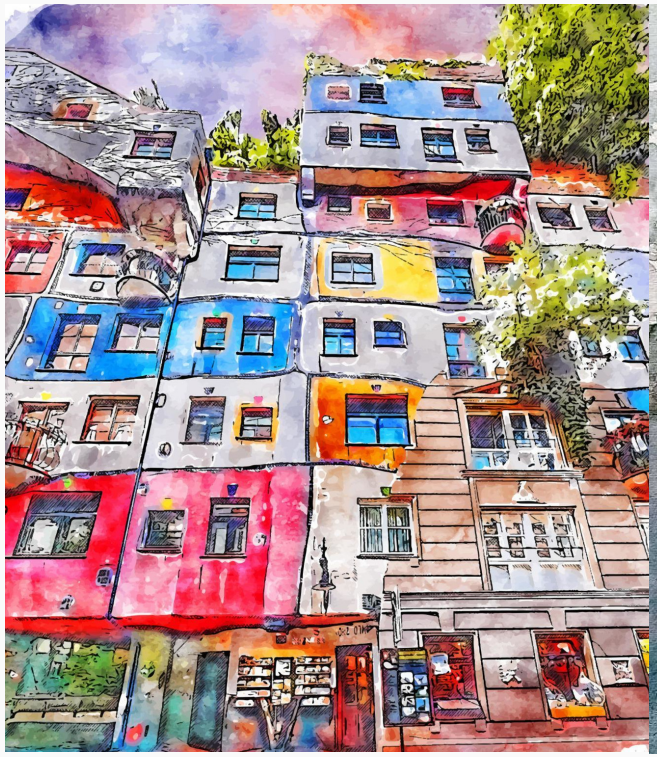


Figure 2: https://de.vecteezy.com/vektorkunst/12911197-wien-osterreich-aquarell-skizze-handgezeichnete-illustration

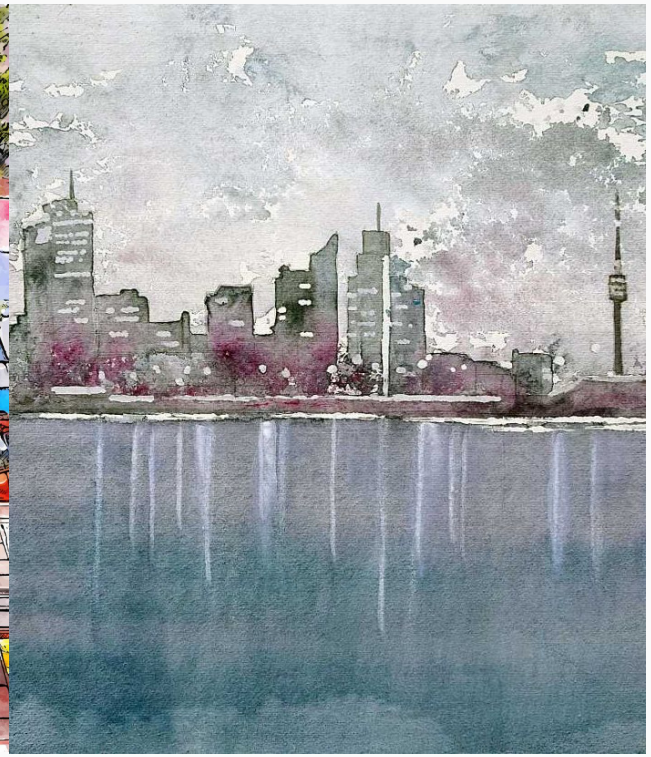
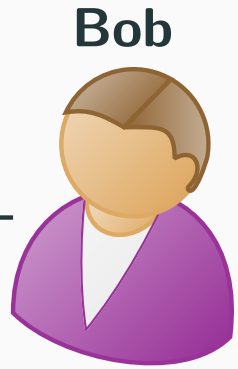


Figure 3: https://www.aquarell.me/bilder/oesterreich/wien

Obliviously rounding two values mod q

Alice



Bob

$$\lfloor a_i + b \rceil \oplus d_{a_i} \quad \forall\, a_i \in \mathbb{Z}_q$$

holds $a \in \mathbb{Z}_q$

learns $\lfloor a + b \rceil$ only

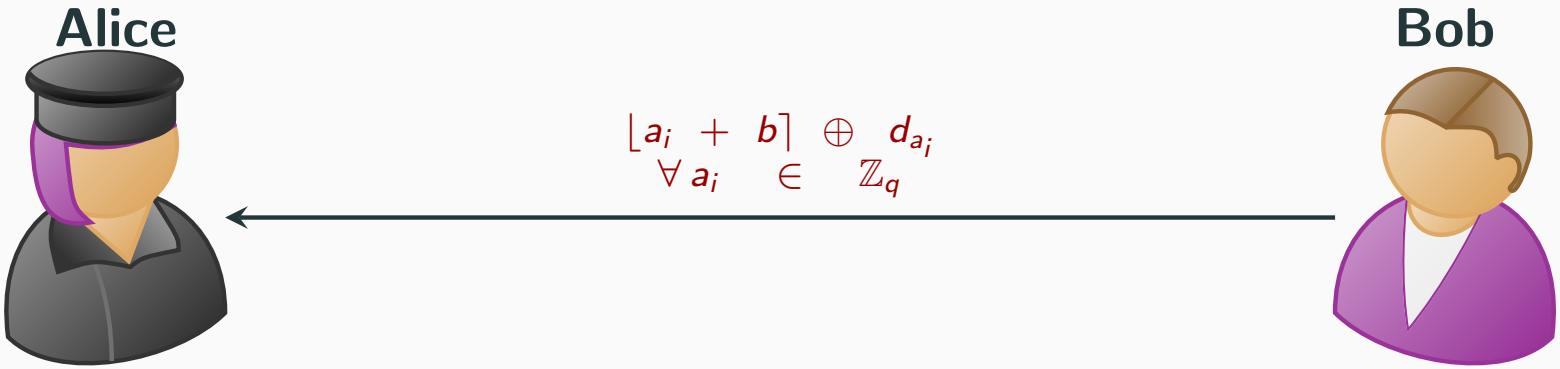
b stays hidden

holds $b \in \mathbb{Z}_q$

learns nothing about

a

Obliviously rounding two values mod q



holds $a \in \mathbb{Z}_q$

learns $\lfloor a + b \rceil$ only
b stays hidden

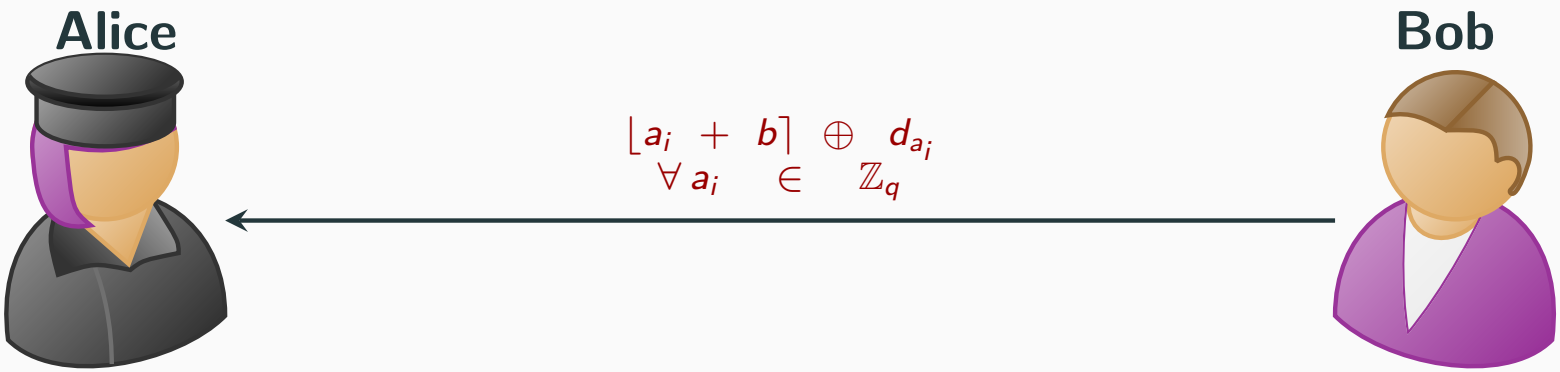
holds $b \in \mathbb{Z}_q$

learns nothing about
a

Example

The first apartment costs 700 Euros to rent.

Obliviously rounding two values mod q



holds $a \in \mathbb{Z}_q$

learns $\lfloor a + b \rceil$ only
b stays hidden

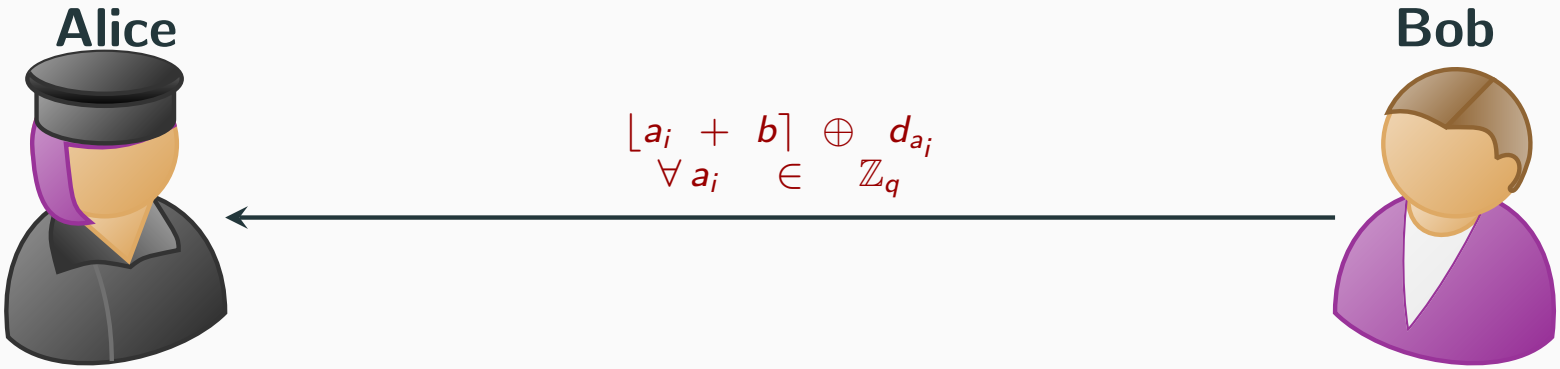
holds $b \in \mathbb{Z}_q$

learns nothing about
a

Example

The first apartment costs 700 Euros to rent. Bob is willing to pay 400. His inputs to the OT are 0, 0, 0, 1, 1, 1, 1, if they agree on 100 Euro steps and staying under 800 Euros per person.

Obliviously rounding two values mod q



holds $a \in \mathbb{Z}_q$

holds $b \in \mathbb{Z}_q$

learns $\lfloor a + b \rceil$ only
b stays hidden

learns nothing about
a

Example

The first apartment costs 700 Euros to rent. Bob is willing to pay 400. His inputs to the OT are 0, 0, 0, 1, 1, 1, 1, if they agree on 100 Euro steps and staying under 800 Euros per person.

Generalizes for small value spaces!

Beyond Simple Arithmetic

Two Telescopes, One Detection

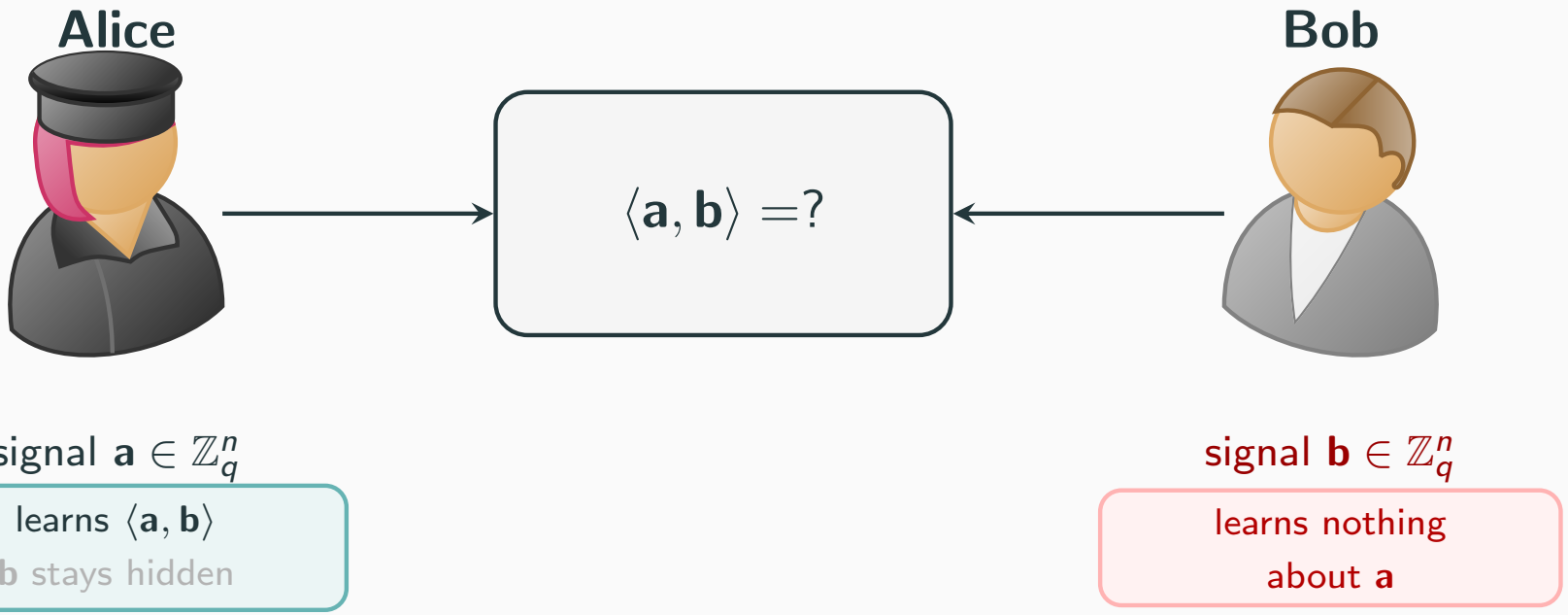
Motivating example

Two radio telescopes record signals $\mathbf{a}, \mathbf{b} \in \mathbb{Z}_q^n$ independently. To confirm a detection, they need the correlation $\langle \mathbf{a}, \mathbf{b} \rangle$, but neither wants to expose their raw signal.

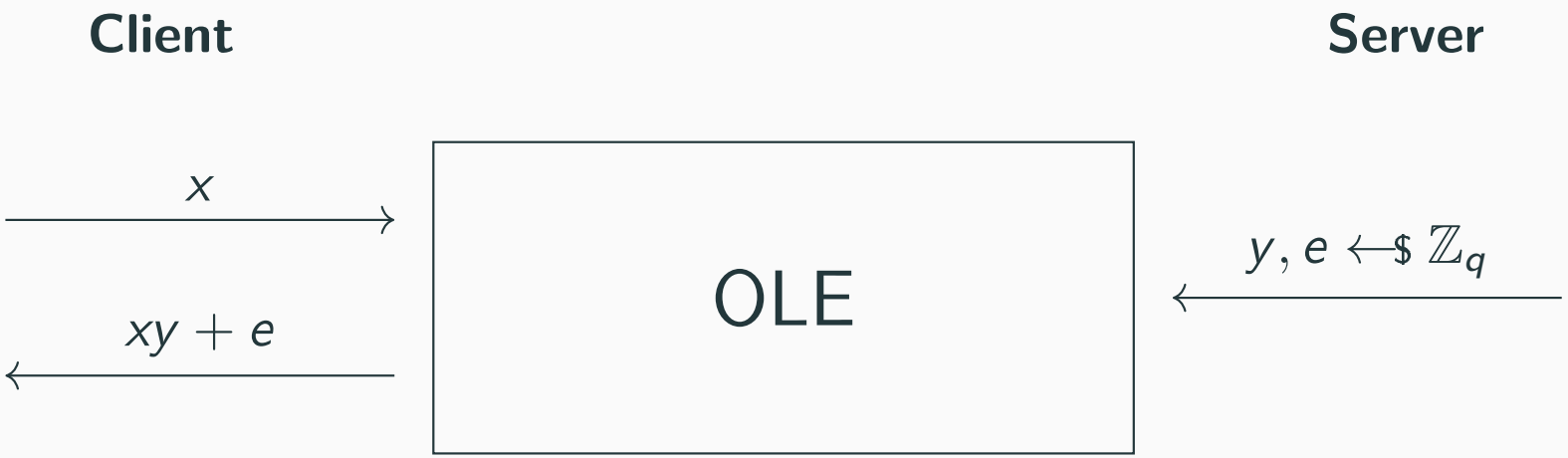
Two Telescopes, One Detection

Motivating example

Two radio telescopes record signals $\mathbf{a}, \mathbf{b} \in \mathbb{Z}_q^n$ independently. To confirm a detection, they need the correlation $\langle \mathbf{a}, \mathbf{b} \rangle$, but neither wants to expose their raw signal.



Oblivious Linear Evaluation (Gilboa [Gil99])



Client learns $xy + e$
Server learns nothing about x
Client learns nothing about y or e

Oblivious Linear Evaluation

- The correlation $\langle \mathbf{a}, \mathbf{b} \rangle = \sum_i a_i b_i$ involves products of private values — hard to compute jointly

Oblivious Linear Evaluation

- The correlation $\langle \mathbf{a}, \mathbf{b} \rangle = \sum_i a_i b_i$ involves products of private values — hard to compute jointly
- OLE converts one such product into additive shares:

$$\text{Alice gets } y_i = a_i b_i + e_i, \qquad \text{Bob keeps } e_i$$

Neither learns the other's input

Oblivious Linear Evaluation

- The correlation $\langle \mathbf{a}, \mathbf{b} \rangle = \sum_i a_i b_i$ involves products of private values — hard to compute jointly
- OLE converts one such product into additive shares:

$$\text{Alice gets } y_i = a_i b_i + e_i, \qquad \text{Bob keeps } e_i$$

  Neither learns the other's input
- Summing over i gives additive shares of the full inner product:

$$\underbrace{\sum_i y_i}_{\text{Alice}} - \underbrace{\sum_i e_i}_{\text{Bob}} = \langle \mathbf{a}, \mathbf{b} \rangle$$

Oblivious Linear Evaluation

- The correlation $\langle \mathbf{a}, \mathbf{b} \rangle = \sum_i a_i b_i$ involves products of private values — hard to compute jointly
- OLE converts one such product into additive shares:

$$\text{Alice gets } y_i = a_i b_i + e_i, \qquad \text{Bob keeps } e_i$$

  Neither learns the other's input
- Summing over i gives additive shares of the full inner product:

$$\underbrace{\sum_i y_i}_{\text{Alice}} - \underbrace{\sum_i e_i}_{\text{Bob}} = \langle \mathbf{a}, \mathbf{b} \rangle$$

- Additive shares are linear: easy to mask, rerandomize, and feed into the next step

How is this useful?

Have you ever wanted to...

- . . . send a hash over a wire?

Have you ever wanted to...

- . . . send a hash over a wire?
- . . . intersect a set privately?

Have you ever wanted to...

- ... send a hash over a wire?
- ... intersect a set privately?
- ... take your low-entropy input and get a high-entropy output?

You may want to consider an OPRF!

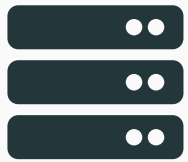
Client

input $x \in \mathcal{X}$, gets y

compute

$$y := F_k(x)$$

Server



key $k \in \mathcal{K}$

Pseudorandom Functions [GGM84, GGM86]

- deterministic and polynomial time function $y = F(x, k)$

Pseudorandom Functions [GGM84, GGM86]

- deterministic and polynomial time function $y = F(x, k)$
- hard to distinguish output y from a randomly chosen element from $y' \in \mathcal{Y}$.

Pseudorandom Functions [GGM84, GGM86]

- deterministic and polynomial time function $y = F(x, k)$
- hard to distinguish output y from a randomly chosen element from $y' \in \mathcal{Y}$.

Naor-Reingold PRF

- Naor-Reingold PRF [NR04]: built from an Abelian group action $*$

Pseudorandom Functions [GGM84, GGM86]

- deterministic and polynomial time function $y = F(x, k)$
- hard to distinguish output y from a randomly chosen element from $y' \in \mathcal{Y}$.

Naor-Reingold PRF

- Naor-Reingold PRF [NR04]: built from an Abelian group action $*$
- Sample $g_0, \ldots, g_m$; compute $g_0 * \prod_{i\,:\,x_i=1} g_i$

Pseudorandom Functions [GGM84, GGM86]

- deterministic and polynomial time function $y = F(x, k)$
- hard to distinguish output y from a randomly chosen element from $y' \in \mathcal{Y}$.

Naor-Reingold PRF

- Naor-Reingold PRF [NR04]: built from an Abelian group action $*$
- Sample $g_0, \ldots, g_m$; compute $g_0 * \prod_{i\,:\,x_i=1} g_i$
- *hashes to group* for free

Pseudorandom Functions [GGM84, GGM86]

- deterministic and polynomial time function $y = F(x, k)$
- hard to distinguish output y from a randomly chosen element from $y' \in \mathcal{Y}$.

Naor-Reingold PRF

- Naor-Reingold PRF [NR04]: built from an Abelian group action $*$
- Sample $g_0, \ldots, g_m$; compute $g_0 * \prod_{i\,:\,x_i=1} g_i$
- *hashes to group* for free

Example (Naor-Reingold with seven input bits)

For example, for 0110111 we would compute the group action $g_0 * g_2 * g_3 * g_5 * g_6 * g_7$.

From Elliptic Curves to Lattices

Elliptic curves

$F_k(x) = \sum_{i\,:\,x_i=1} P_i$

Algebraic structure
hidden by ECDLP
✓Broken in presence of
a quantum adversary

From Elliptic Curves to Lattices

Elliptic curves

$F_k(x) = \sum_{i\,:\,x_i=1} P_i$

Algebraic structure hidden by ECDLP

✓Broken in presence of a quantum adversary

Subset products

$F_k(x) = \prod_{i\,:\,x_i=1} g_i \mod p$

Broken: multiplicative structure is exploitable

From Elliptic Curves to Lattices

Elliptic curves

$F_k(x) = \sum_{i\,:\,x_i=1} P_i$

Algebraic structure hidden by ECDLP

✓Broken in presence of a quantum adversary

Subset products

$F_k(x) = \prod_{i\,:\,x_i=1} g_i \mod p$

Broken: multiplicative structure is exploitable

Rounded products

$F_k(x) = \left\lfloor \prod_{i\,:\,x_i=1} g_i \right\rceil \mod q$

Post-quantum ✓

. . . needs careful rounding!

From Elliptic Curves to Lattices

Elliptic curves

$F_k(x) = \sum_{i\,:\,x_i=1} P_i$

Algebraic structure hidden by ECDLP

✓Broken in presence of a quantum adversary

Subset products

$F_k(x) = \prod_{i\,:\,x_i=1} g_i \mod p$

Broken: multiplicative structure is exploitable

Rounded products

$F_k(x) = \left\lfloor \prod_{i\,:\,x_i=1} g_i \right\rceil \mod q$

Post-quantum ✓

. . . needs careful rounding!

Rounding kills the algebraic structure, but how do we round *obliviously*?

Spring is a subset-product-based PRF

- 2014: heuristic PRF with using polynomial subset-products with small modulus

Spring is a subset-product-based PRF

- 2014: heuristic PRF with using polynomial subset-products with small modulus
  $q \in \{257, 514\}$

Spring is a subset-product-based PRF

- 2014: heuristic PRF with using polynomial subset-products with small modulus
  $q \in \{257, 514\}$
- non-heuristic variant: huge q!

Spring is a subset-product-based PRF

- 2014: heuristic PRF with using polynomial subset-products with small modulus
  $q \in \{257, 514\}$
- non-heuristic variant: huge q!
  but we're working on it

Spring is a subset-product-based PRF

- 2014: heuristic PRF with using polynomial subset-products with small modulus
  $q \in \{257, 514\}$
- non-heuristic variant: huge q!
  but we're working on it
- [HKL+25] focus in SPRING-BCH: rounding bias reduction using extended BCH code $[128, 64, 22]$

Spring is a subset-product-based PRF

- 2014: heuristic PRF with using polynomial subset-products with small modulus
  $q \in \{257, 514\}$
- non-heuristic variant: huge q!
  but we're working on it
- [HKL+25] focus in SPRING-BCH: rounding bias reduction using extended BCH code $[128, 64, 22]$

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\mathbf{k}_0 \prod_{i=1}^{m} \mathbf{k}_i^{c_i}\right)$$

Spring is a subset-product-based PRF

- 2014: heuristic PRF with using polynomial subset-products with small modulus
  $q \in \{257, 514\}$
- non-heuristic variant: huge q!
  but we're working on it
- [HKL+25] focus in SPRING-BCH: rounding bias reduction using extended BCH code $[128, 64, 22]$

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\mathsf{iNTT}\left(\mathbf{k}_0 \odot \prod_{i=1}^{m} \mathbf{k}_i^{c_i}\right)\right)$$

Spring is a subset-product-based PRF

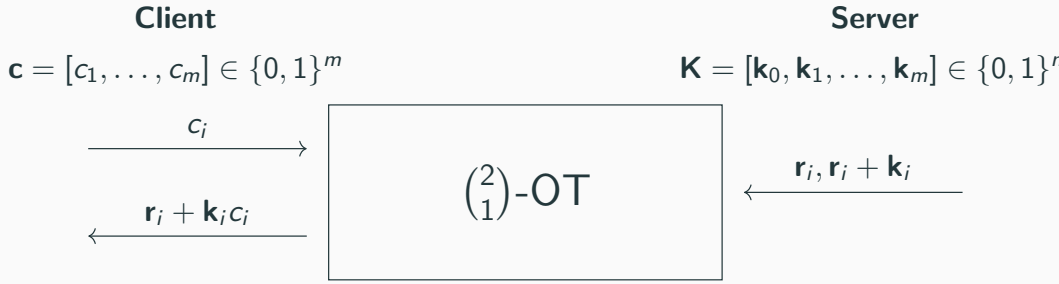
- 2014: heuristic PRF with using polynomial subset-products with small modulus
  $q \in \{257, 514\}$
- non-heuristic variant: huge q!
  but we're working on it
- [HKL+25] focus in SPRING-BCH: rounding bias reduction using extended BCH code $[128, 64, 22]$

$$\mathcal{F}_{\mathbf{K}}(c_1, \cdots, c_m) = S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

Multiplication is addition of exponents!

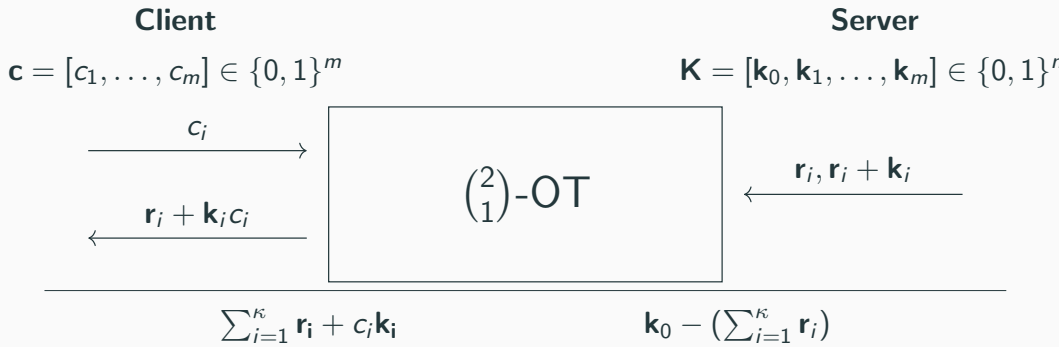
$$g^a g^b = g^{a+b} \text{ for some generator } g$$

An OPRF from Spring



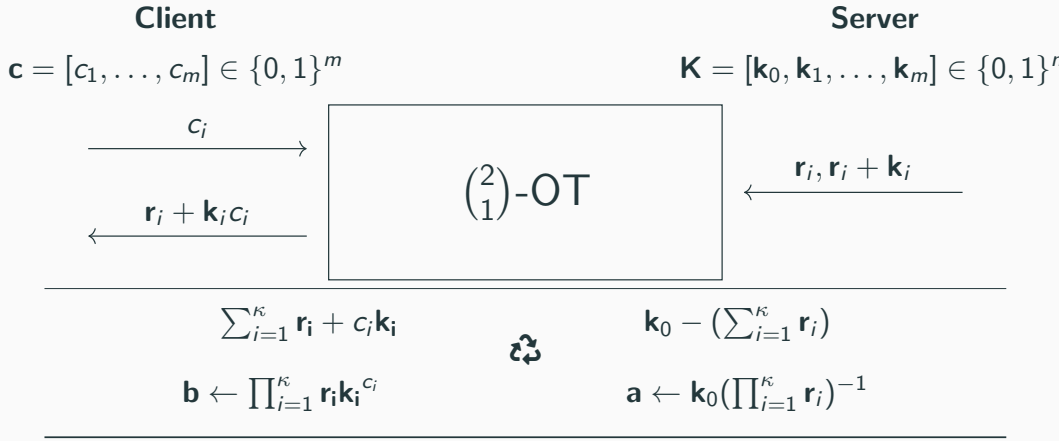
$$S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

An OPRF from Spring



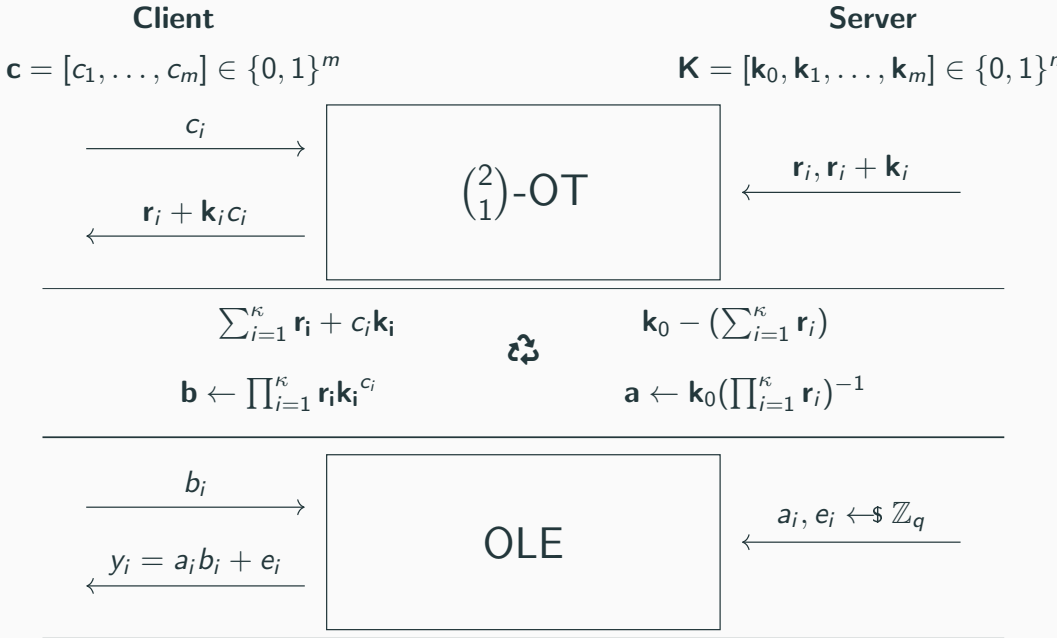
$$S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

An OPRF from Spring



$$S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

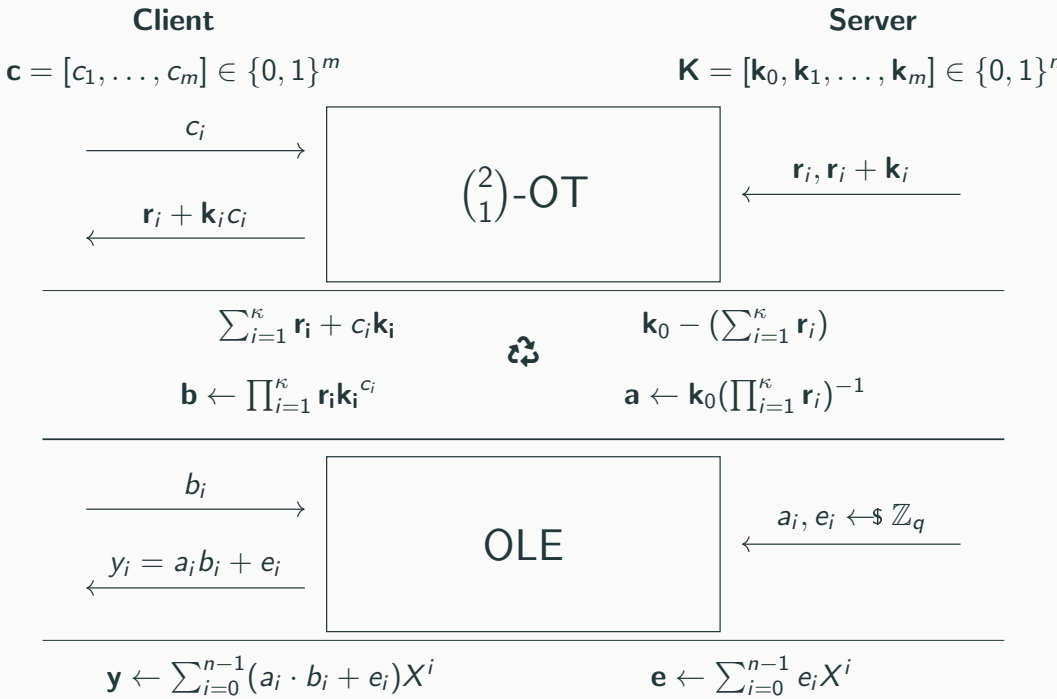
An OPRF from Spring



$$S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$

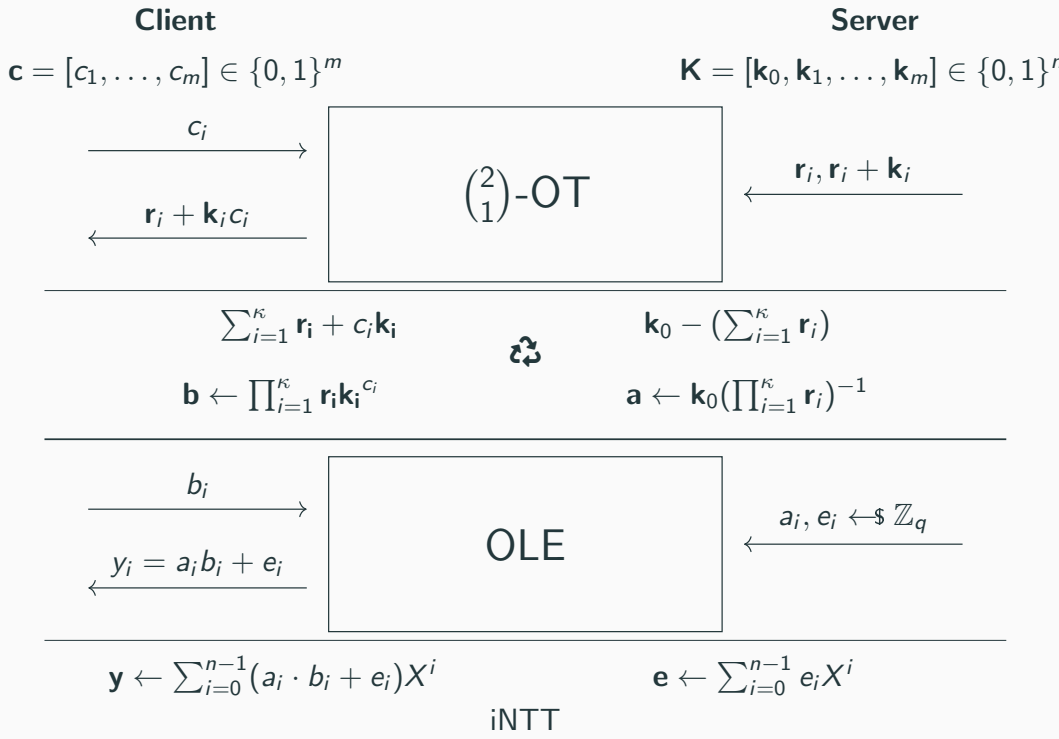
An OPRF from Spring

$$S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$



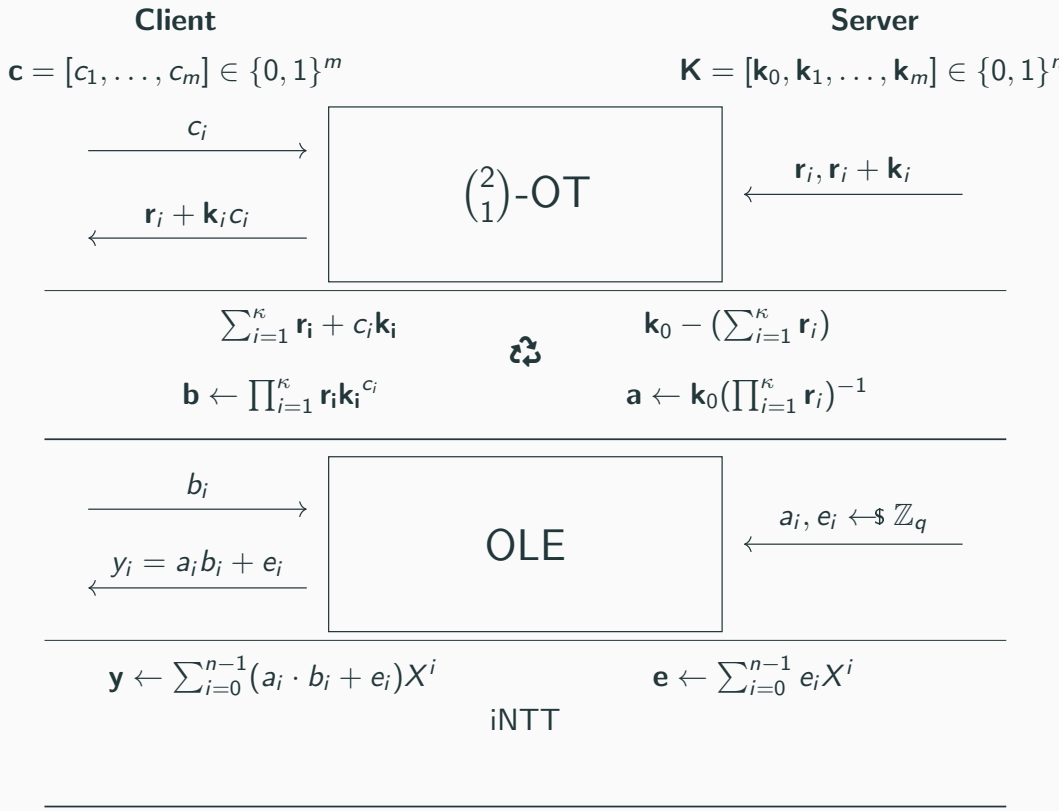
An OPRF from Spring

$$S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$



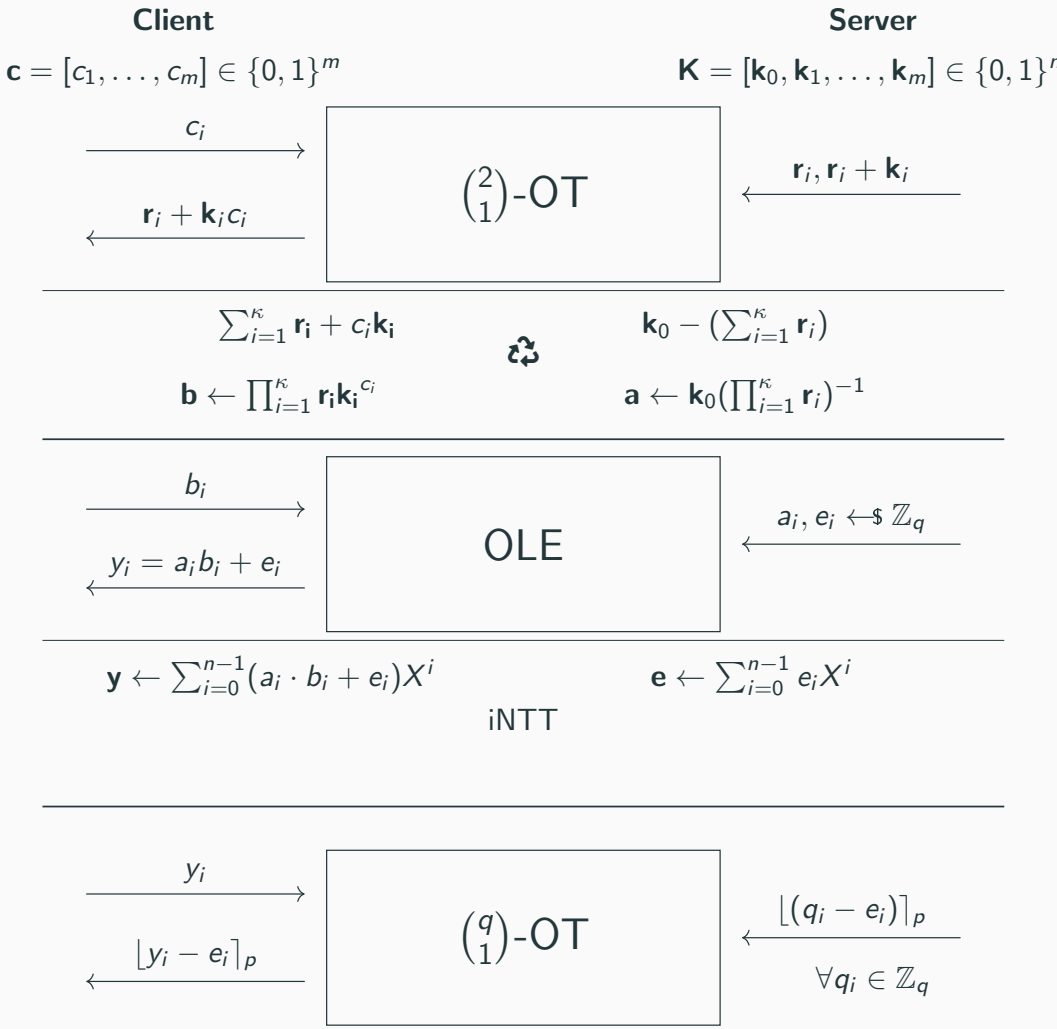
An OPRF from Spring

$$S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$



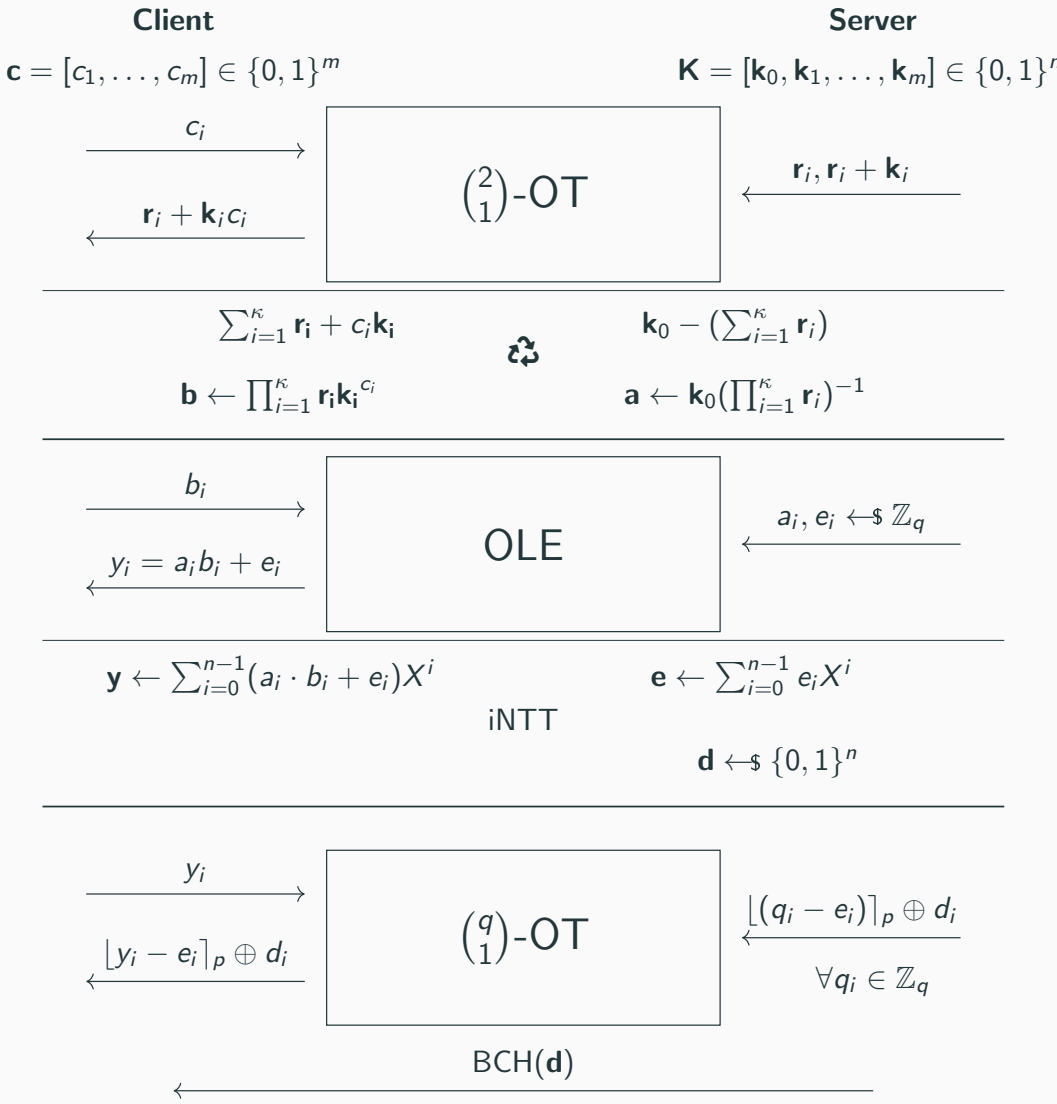
An OPRF from Spring

$$S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$



An OPRF from Spring

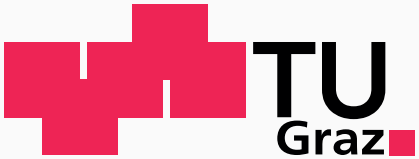
$$S\left(\text{iNTT}\left(\text{lookup}\left(\mathbf{k}_0 + \sum_{i=1}^{m} c_i \mathbf{k}_i\right)\right)\right)$$



Conclusion

- Generic primitives, composed carefully, give us efficient and provably secure protocols
- Ongoing: tighter reductions, smaller parameters, stronger security arguments
- Many open problems- happy to discuss!

lena.heimberger@tugraz.at



Oblivious Transfer, Extended

Oblivious Evaluations for Privacy

Lena Heimberger

QSCAN 2026, Vienna

9^{th} of June, 2026

References i


Oded Goldreich, Shafi Goldwasser, and Silvio Micali.
On the cryptographic applications of random functions.
pages 276–288, 1984.


Oded Goldreich, Shafi Goldwasser, and Silvio Micali.
How to construct random functions.
Journal of the ACM, 33(4):792–807, October 1986.


Niv Gilboa.
Two party RSA key generation.
pages 116–129, 1999.


Lena Heimberger, Tobias Hennerbichler, Fredrik Meisingseth, Sebastian Ramacher, and Christian Rechberger.
OPRFs from isogenies: Designs and analysis.
2024.

References ii


Lena Heimberger, Daniel Kales, Riccardo Lolato, Omid Mir, Sebastian Ramacher, and Christian Rechberger.
Leap: A fast, lattice-based OPRF with application to private set intersection.
pages 254–283, 2025.


Daniel Kales, Christian Rechberger, Thomas Schneider, Matthias Senker, and Christian Weinert.
Mobile private contact discovery at scale.
pages 1447–1464, 2019.


Moni Naor and Omer Reingold.
Number-theoretic constructions of efficient pseudo-random functions.
Journal of the ACM, 51(2):231–262, March 2004.

All OT is precomputed

#	Protocol	Communication		Computation	
		Client	Server	Client	Server
2^0	Simplest OT+IKNP	39 kB	**4 kB**	63 ms	63 ms
	Kyber OT+IKNP	465 kB	328 kB	**10 ms**	**10 ms**
	Simplest OT+Silent	**22 kB**	22 kB	68 ms	68 ms
	Kyber OT+Silent	448 kB	392 kB	**10 ms**	10 ms
2^{13}	Simplest OT+IKNP	319 MB	**4.26 kB**	420 ms	463 ms
	Kyber OT+IKNP	319 MB	328 kB	**311 ms**	**423 ms**
	Simplest OT+Silent	**46 kB**	155 kB	2065 ms	3371 ms
	Kyber OT+Silent	487 kB	559 kB	2130 ms	3496 ms

All OT is precomputed

#	Protocol	Communication		Computation	
		Client	Server	Client	Server
2^0	Simplest OT+IKNP	39 kB	**4 kB**	63 ms	63 ms
	Kyber OT+IKNP	465 kB	328 kB	**10 ms**	**10 ms**
	Simplest OT+Silent	**22 kB**	22 kB	68 ms	68 ms
	Kyber OT+Silent	448 kB	392 kB	**10 ms**	10 ms
2^{13}	Simplest OT+IKNP	319 MB	**4.26 kB**	420 ms	463 ms
	Kyber OT+IKNP	319 MB	328 kB	**311 ms**	**423 ms**
	Simplest OT+Silent	**46 kB**	155 kB	2065 ms	3371 ms
	Kyber OT+Silent	487 kB	559 kB	2130 ms	3496 ms

The OT is interchangable!

Performance

Phase	Client			Server		
	Comm.	Comp.	Idle	Comm.	Comp.	Idle
Subset-Sum	16 bytes	5 μs	21 μs	16 384 bytes	863 μs	46 μs
OLE	144 bytes	4 μs	3 μs	1 296 bytes	72 μs	88 μs
Rounding	144 bytes	2 μs	726 μs	4 118 bytes	814 μs	67 μs
BCH	0 bytes	1 μs	0 μs	8 bytes	/	26 μs
Overall	304 bytes	11 μs	750 μs	21 806 bytes	1.7 ms	227 μs
(Network)	(328 bytes)			(22 840 bytes)		

	Parameters		Setup		Online	
	$\lvert S\rvert$	$\lvert C\rvert$	$\lvert S\rvert$	$\lvert C\rvert$	$\lvert S\rvert$	$\lvert C\rvert$
Leap (IKNP+Kyber)	2^0	2^0	1 ms 117 bytes	1 ms 0 bytes	0.2s 73.4 kiB	6 ms 20 KiB
	2^5	2^5	1 ms 246 bytes	1 ms 0 bytes	0.08 s 802 kiB	0.07 s 47.5 kiB
	2^{10}	2^{10}	4 ms 4.30 kiB	5 ms 0 bytes	2.4 s 22.39 MiB	2.46 s 646.5 kiB
NR-OT (FHE OT) [HHM+24]	2^0	2^0	0.26 s 134 bytes	0.51 s 1 byte	0.06 s 128 kiB	0.10 s 0.75MiB
	2^5	2^5	1.63 s 263 bytes	1.88 s 1 byte	3.11 s 4MiB	3.15 s 8.5 MiB
	2^{10}	2^{10}	45.04 s 4.31 MiB	45.28 s 1 byte	99.66 s 128 MiB	99.71 s 256.6 MiB
OPUS [HHM+24]	2^0	2^0	0.26 s 133 bytes	0.26 s 0 bytes	15.47 s 17.07 kiB	15.91 s 9.04 kiB
	2^5	2^5	8.71 s 262 bytes	8.71 s 0 bytes	328.46 s 546.25 kiB	329.14 s 290.26 kiB
	2^{10}	2^{10}	303.38 s 4.31 kiB	303.38 s 0 bytes	16367.12 s 34.14 MiB	16367.60 s 18.08 MiB
ECNR (Simplest OT + IKNP) [KRS+19]	2^0	2^0	10 ms 133 bytes	0 s 0 bytes	0.23 s 12.04 kiB	0.05 s 16 bytes
	2^5	2^5	0.02 s 262 bytes	0 s 0 bytes	0.21 s 137.05 kiB	0.06 s 512 bytes
	2^{10}	2^{10}	0.3 s 4.36 kiB	0 s 0 bytes	0.64 s 4.04 MiB	0.57 s 16 kiB